

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

**Практическая работа
“Д33: Тестирование API средствами Postman”**

Выполнил:

Данилова Анастасия

Группа

К3339

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

1. реализовать тестирование API средствами Postman;
2. написать тесты внутри Postman.

Ход работы

В рамках работы было реализовано тестирование REST API сервиса аренды недвижимости с использованием приложения Postman.

Целью тестирования являлась проверка основного пользовательского сценария взаимодействия с системой.

Для тестирования был создан единый комплексный сценарий (Main Flow), включающий последовательное выполнение запросов к API и автоматическую проверку корректности ответов с помощью встроенных тестов Postman.

Реализованный сценарий

В ходе работы был реализован следующий пользовательский сценарий:

1. Авторизация владельца недвижимости (LANDLORD);
2. Получение списка собственных объявлений;
3. Авторизация арендатора (TENANT);
4. Получение списка всех объявлений;
5. Фильтрация объявлений по параметрам:
тип недвижимости — APARTMENT;\nгород — Moscow;
6. Просмотр информации о случайно выбранном объявлении;
7. Создание чата по выбранному объявлению.

Структура тестирования

Для хранения промежуточных данных использовались environment variables Postman:

Переменная	Назначение
baseUrl	базовый URL API
landlordToken	JWT токен владельца
tenantToken	JWT токен арендатора
propertyId	идентификатор выбранного объявления
chatId	идентификатор созданного чата

Search collections

Rental Service API - Main Flow

LANDLORD: Login and view my pro...

POST

1) Login as LANDLORD

GET

2) View my properties (as LAND...

TENANT: Search and start chat

POST

3) Login as TENANT

GET

4) List all properties

GET

5) Filter properties (APARTMEN...

GET

6) View random property details

POST

7) Start chat about this property

Run Sequence

Deselect All | Select All | Reset

1

>

POST

1) Login as LANDLORD

2

>

GET

2) View my properties (as LANDLOR

3

>

POST

3) Login as TENANT

4

>

GET

4) List all properties

5

>

GET

5) Filter properties (APARTMENT in

6

>

GET

6) View random property details

7

>

POST

7) Start chat about this property

Functional

Performance

Choose how to run your collection

Run manually

Run this collection in the Collection Runner.

Schedule runs

Periodically run collection at a specified time on the Postman Cloud.

Automate runs via CLI

Configure CLI command to run on your build pipeline.

Run configuration

Iterations 1

Delay 0 ms

Test data file

Only JSON and CSV files are accepted.

Select File

Advanced settings

Run Rental Service API - Main FL...

Search collections

Rental Service API - Main Flow

LANDLORD: Login and view my pro...

POST

1) Login as LANDLORD

GET

2) View my properties (as LAND...

TENANT: Search and start chat

POST

3) Login as TENANT

GET

4) List all properties

GET

5) Filter properties (APARTMEN...

GET

6) View random property details

POST

7) Start chat about this property

Rental Service API - Main Flow - Run results

Run today at 10:24:13 PM · View all runs

Source

Environment

Iterations

Duration

All tests

Errors

Avg. Resp. Time

Runner

Rental Service API (local)

1

1s 266ms

18

0

32 ms

All Tests Passed (18) Failed (0) Skipped (0) Errors (0) Console Log

View Summary

Iteration 1

POST

LANDLORD: Login and view my properties / 1) Login as LANDLORD

http://localhost:5000/auth/login

PASS: Status is 200

PASS: Has token

200 · 126 ms · 451 B · 2

GET

LANDLORD: Login and view my properties / 2) View my properties (as LANDLORD)

http://localhost:5000/me/properties

PASS: Status is 200

PASS: Returns array of properties

PASS: First property has required fields

PASS: Has usable landlord token variable

200 · 6 ms · 564 B · 4

POST

TENANT: Search and start chat / 3) Login as TENANT

http://localhost:5000/auth/login

PASS: Status is 200

PASS: Has token

200 · 79 ms · 446 B · 2

GET

TENANT: Search and start chat / 4) List all properties

http://localhost:5000/properties

PASS: Status is 200

PASS: Returns array

PASS: Has at least one property

200 · 3 ms · 564 B · 3

GET

TENANT: Search and start chat / 5) Filter properties (APARTMENT in Moscow)

http://localhost:5000/properties?type=APARTMENT&city=Moscow

PASS: Status is 200

PASS: Returns filtered array

PASS: All properties match filters

200 · 5 ms · 564 B · 3

GET

TENANT: Search and start chat / 6) View random property details

http://localhost:5000/properties/1

PASS: Status is 200

PASS: Returns full property details

PASS: Property has monthly price

200 · 4 ms · 562 B · 3

POST

TENANT: Search and start chat / 7) Start chat about this property

http://localhost:5000/properties/1/chats

PASS: Status is 200 or 201

200 · 4 ms · 676 B · 1

Реализованные проверки

В каждом запросе были реализованы автоматические тесты.

Проверка авторизации

Проверялись:

- успешный HTTP-статус;
- наличие JWT-токена;
- сохранение токена в переменные окружения.

Проверка списка объявлений

Проверялось:

- что сервер возвращает массив;
- наличие обязательных полей:
 - id
 - title
 - owner_id

Проверка фильтрации

После фильтрации выполнялась проверка, что все объявления соответствуют заданным параметрам.

Проверка просмотра объявления

Проверялось:

- совпадение идентификатора объявления;
- наличие описания;
- наличие стоимости аренды.

Проверка создания чата

Проверялись:

- HTTP-статус 200 или 201;
- наличие идентификатора чата;
- корректное сохранение chatId.

Вывод

В ходе работы было реализовано комплексное тестирование REST API системы аренды недвижимости средствами Postman.

Был разработан полный пользовательский сценарий взаимодействия с системой, включающий авторизацию пользователей, работу со списком объявлений, фильтрацию недвижимости, просмотр детальной информации и создание чата между арендатором и владельцем недвижимости.

С помощью встроенных тестов Postman была реализована автоматическая проверка корректности ответов сервера, структуры JSON и логики взаимодействия между запросами.

В результате была получена полностью работоспособная коллекция тестов, позволяющая автоматически проверять основной функционал API приложения.